

SALESFORCE PROJECT IMPLEMENTATION PHASES

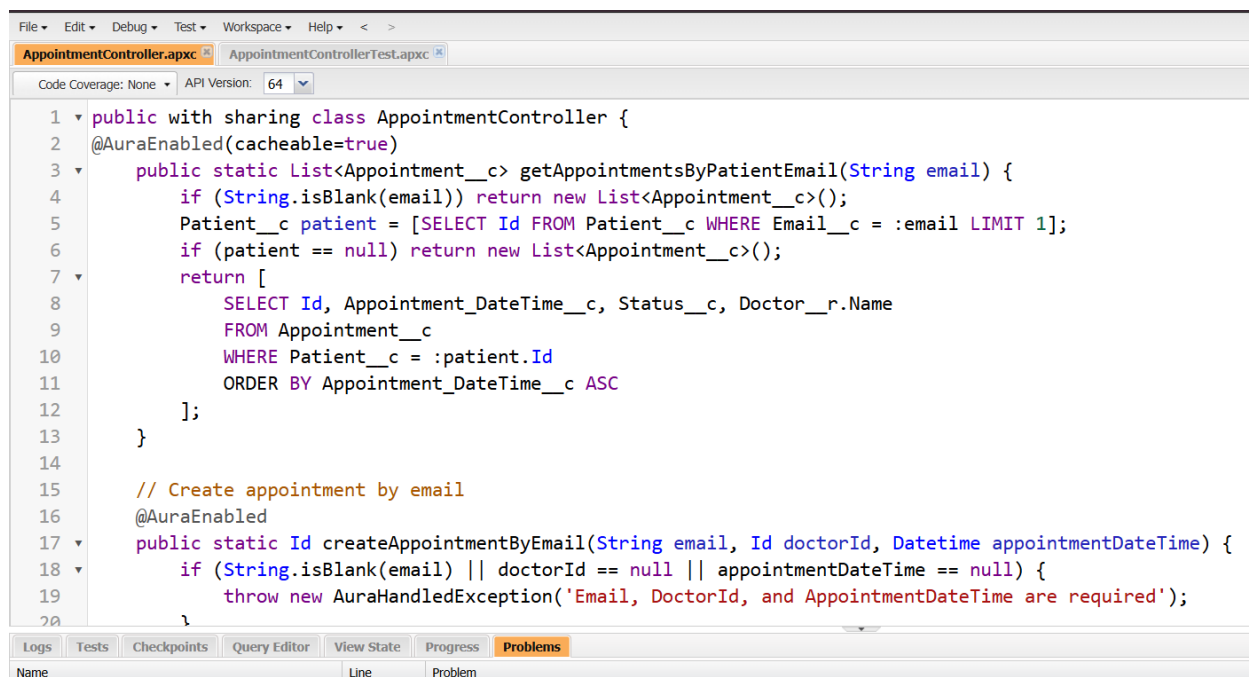
MediSmart CRM – AI-Powered Healthcare Appointment & Patient Follow-up CRM

Phase 5: Process Automation (Admin)

AppointmentController.apxc

This Apex controller serves as the backend logic for the Clinic Chatbot Lightning Web Component, handling all data operations related to appointments.

- **Primary Purpose:** To expose appointment data securely to the frontend chatbot interface, allowing users to view and create appointments.
- **AuraEnabled Methods:** The class uses the `@AuraEnabled` annotation, making its methods callable directly from the Lightning component. This is the critical link between the chatbot's UI and Salesforce's backend.
- **Functionality:** Show Appointment, Book Appointment, Cancel and reschedule appointment

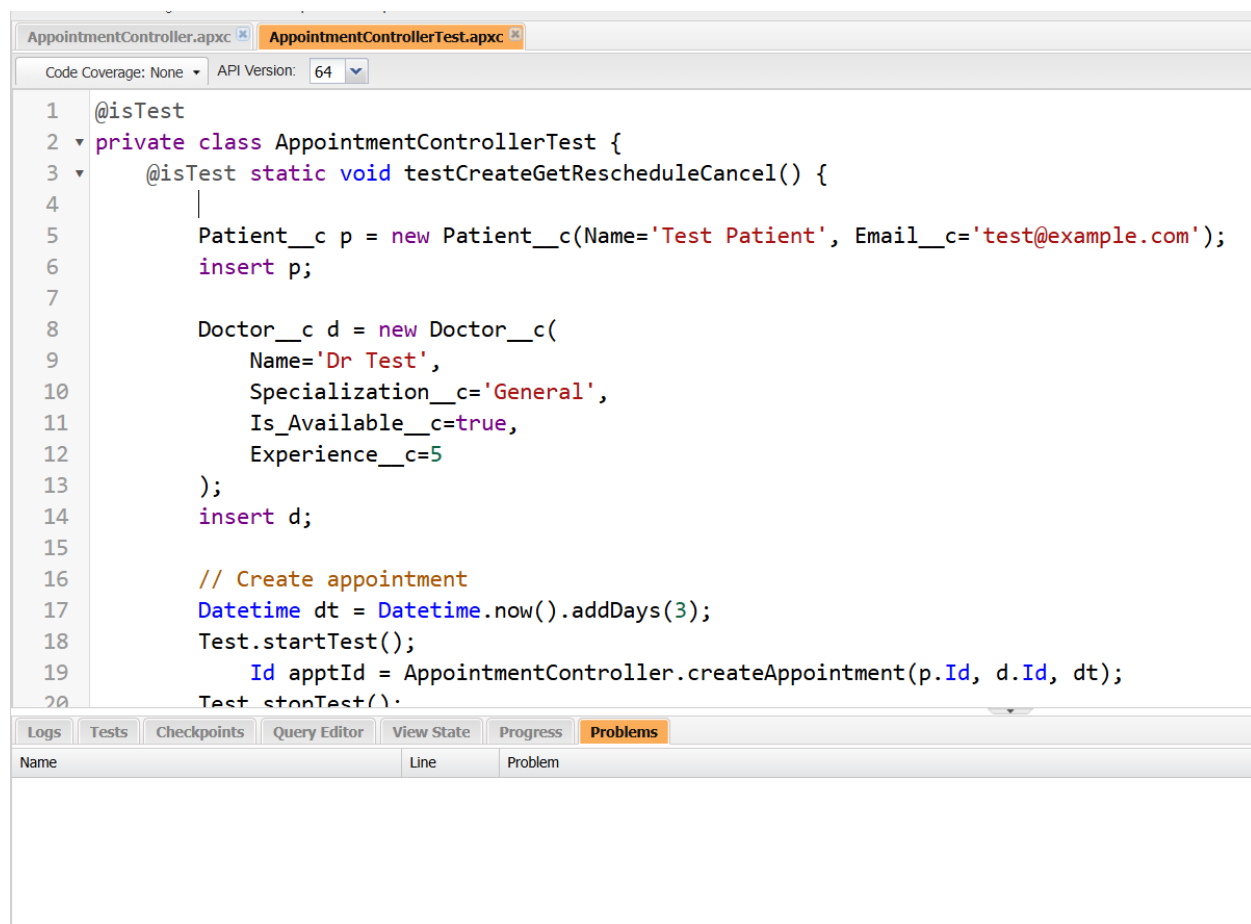


```
1 public with sharing class AppointmentController {
2     @AuraEnabled(cacheable=true)
3     public static List<Appointment__c> getAppointmentsByPatientEmail(String email) {
4         if (String.isBlank(email)) return new List<Appointment__c>();
5         Patient__c patient = [SELECT Id FROM Patient__c WHERE Email__c = :email LIMIT 1];
6         if (patient == null) return new List<Appointment__c>();
7         return [
8             SELECT Id, Appointment_DateTime__c, Status__c, Doctor__r.Name
9             FROM Appointment__c
10            WHERE Patient__c = :patient.Id
11            ORDER BY Appointment_DateTime__c ASC
12        ];
13    }
14
15    // Create appointment by email
16    @AuraEnabled
17    public static Id createAppointmentByEmail(String email, Id doctorId, Datetime appointmentDateTime) {
18        if (String.isBlank(email) || doctorId == null || appointmentDateTime == null) {
19            throw new AuraHandledException('Email, DoctorId, and AppointmentDateTime are required');
20        }
21    }
```

Apex Test Class: **AppointmentControllerTest.apxc**

This test class ensures the reliability and correctness of the AppointmentController.

- Purpose: To verify that all methods in the controller work as expected by simulating real-world scenarios without altering live data.
- Test Data Creation: The test method first creates necessary prerequisite records (Patient, Doctor) to provide a realistic testing environment for the controller's logic.
- Method Verification: It directly calls the createAppointment method from the controller to confirm that an appointment record can be successfully created with the test data provided. This validates the core "booking" functionality of the chatbot.



```
1  @isTest
2  private class AppointmentControllerTest {
3      @isTest static void testCreateGetRescheduleCancel() {
4          |
5          Patient__c p = new Patient__c(Name='Test Patient', Email__c='test@example.com');
6          insert p;
7          |
8          Doctor__c d = new Doctor__c(
9              Name='Dr Test',
10             Specialization__c='General',
11             Is_Available__c=true,
12             Experience__c=5
13         );
14         insert d;
15         |
16         // Create appointment
17         Datetime dt = Datetime.now().addDays(3);
18         Test.startTest();
19         Id apptId = AppointmentController.createAppointment(p.Id, d.Id, dt);
20         Test.stopTest();
21     }
```

Name	Line	Problem
------	------	---------